

Análisis de herramientas para Vibecoding

Leonardo Baranda, Pablo Camean, Martín Tigani, Ignacio Collazo, Gabriel Arias
Universidad Nacional de La Matanza
Buenos Aires, Argentina
{lbaranda, pcamean, mtigani, icollazo, gaarias}@alumno.unlam.edu.ar

Resumen

En los últimos años, la Inteligencia Artificial Generativa (IAG) empezó a ocupar un lugar central en el desarrollo de software, ya que permite crear programas a partir de descripciones en lenguaje natural. En este marco aparece el concepto de Vibecoding, entendido como la delegación de la implementación técnica en modelos de lenguaje avanzados (LLMs) mediante prompts conversacionales. En el presente trabajo se llevó a cabo una experiencia práctica orientada a la generación de un sistema cliente-servidor concurrente en Python, compuesto por un servidor de juego de preguntas y un cliente con interfaz gráfica (GUI) desarrollado en Tkinter. El proceso consistió en diseñar un prompt técnico, seguido de iteraciones correctivas, hasta obtener código funcional capaz de gestionar varios jugadores en paralelo, utilizando sockets, hilos y mecanismos de sincronización [1][2][3]. Se realizó un análisis crítico del código generado, identificando sus fortalezas y debilidades. Los resultados preliminares muestran que la herramienta de Vibecoding logra producir soluciones concurrentes ejecutables tras ajustes iterativos, lo que abre la puerta a la investigación enfocada en aplicar métricas de complejidad ciclomática, mantenibilidad y eficiencia de concurrencia, con el fin de evaluar rigurosamente la calidad del software.

Keywords: Inteligencia Artificial Generativa, Prompt, Vibecoding, Métricas de programación, concurrencia.

1. Introducción

La Inteligencia Artificial (IA) ha transformado radicalmente múltiples prácticas profesionales y científicas; en el ámbito del desarrollo de software, las técnicas recientes de IA Generativa permiten ya producir fragmentos de código a partir de descripciones en lenguaje natural, modificando la dinámica tradicional de creación y verificación del software. Herramientas y asistentes basados en modelos de lenguaje han pasado de ser solo aliados para completar líneas de código a convertirse en agentes que participan activamente en el proceso de desarrollo, reconfigurando roles y flujos de trabajo en equipos de ingeniería. Este cambio plantea interrogantes no sólo sobre productividad y accesibilidad, sino también sobre la calidad, la trazabilidad y la responsabilidad en soluciones programadas con asistencia algorítmica.

Dentro de este panorama emergente, el paradigma conocido como Vibecoding propone delegar la implementación técnica —o gran parte de ella— a modelos de lenguaje mediante prompts técnicos estructurados. Si bien Vibecoding promete acelerar prototipado y disminuir la barrera de entrada para tareas complejas, su aplicación en dominios donde la eficiencia y la robustez son críticas —como la programación concurrente— exige un análisis riguroso. La concurrencia introduce retos particulares (condiciones de carrera, interbloqueos, ordenaciones temporales impredecibles) [4] que complican la validación automática y aumentan el riesgo de fallos difíciles de detectar mediante pruebas funcionales superficiales.

Frente a esta situación, resulta imprescindible complementar la verificación empírica con métricas de ingeniería de software capaces de cuantificar aspectos estructurales y de calidad del código producido por IAG. Métricas como la complejidad ciclomática, el acoplamiento, la cohesión y los índices de mantenibilidad permiten evaluar, desde una perspectiva formal, la propensión a errores, la dificultad de prueba y el coste de evolución del software generado automáticamente. [5] Asimismo, la evaluación de parámetros operativos —por ejemplo, la eficiencia en la sincronización y el comportamiento bajo carga— aporta una dimensión necesaria para juzgar la adecuación de los artefactos generados en escenarios reales de ejecución concurrente.

El presente trabajo aborda precisamente esta intersección: se documenta la generación por Vibecoding de un sistema cliente-servidor concurrente en Python (un servidor multicliente que gestiona un juego de preguntas y un cliente gráfico con interfaz) y se realiza un análisis crítico apoyado en métricas de calidad y pruebas controladas. A través de prompts técnicos estandarizados se produjo y refinó el código objeto de estudio; posteriormente se aplicaron métricas para identificar fortalezas, limitaciones y riesgos inherentes al uso de IAG en programación concurrente. Con estos elementos, el trabajo pretende evidenciar la idoneidad de Vibecoding en contextos académicos y formativos, y proponer criterios de evaluación más precisos para su adopción segura y responsable.

2. Desarrollo

El presente desarrollo toma como caso de estudio una aplicación de juego de preguntas y respuestas generada a partir de un prompt técnico detallado procesado por un modelo de lenguaje avanzado (ChatGPT-5). El código resultante, compuesto por un servidor (servidor.py) y un cliente (cliente.py), sirve como un ejemplo práctico para analizar la implementación de conceptos de programación concurrente y los desafíos vinculados a su diseño.

2.1 Herramienta

Para la generación automática del código base de la aplicación, la herramienta seleccionada fue ChatGPT, desarrollado por OpenAI, basado en la arquitectura GPT-5. Se eligió esta herramienta por su capacidad para interpretar requisitos técnicos complejos y generar código fuente funcional en un lenguaje de programación específico.

2.2 Formato de prompt

Según la clasificación realizada en la tabla 1 del paper [6] se seleccionó el formato de prompt técnico Prompt Pattern, el cual, consiste en una plantilla reutilizable que permitió estructurar en el prompt la solución que se necesita. Este formato, además, cuenta con una serie de patrones que permiten enmarcar el prompt técnico bajo distintas perspectivas. En esta oportunidad, se seleccionó el patrón Input Semantics, el cual, permitió definir el lenguaje a utilizar.

2.3 Definición preliminar de requisitos

A fines de cumplir con los objetivos de esta investigación, definimos los siguientes requisitos:

“Se solicita desarrollar un juego de preguntas y respuestas con Python, el cual debe utilizar comunicación a través de socket y el patrón productor-consumidor para aplicar concurrencia. Su pantalla principal debe contener un input de texto para introducir un nombre o alias, y luego un botón para entrar. Una vez que el usuario registra su nombre o alias e ingresa, deberá pasar a una sala de espera donde se aguarda por más jugadores, superado cierto tiempo, inicia el juego.

La siguiente pantalla estará dada por la pregunta junto con sus 4 posibles respuestas. Esta misma se repetirá por la cantidad de preguntas que se realicen. Finalmente, la última pantalla estará dada por el podio compuesto por los 3 usuarios con mayor puntaje acumulado. El máximo de usuarios que pueden ingresar al sistema de manera simultánea son 10. El juego comienza al alcanzar los 10 usuarios, o bien, si no llega a 10 usuarios, se establece un tiempo de espera. Cuando comienza el juego, a todos los usuarios involucrados se le presenta la misma pregunta donde deberán seleccionar una respuesta de 4 existentes.

La cantidad de preguntas a responder serán 5 y cada pregunta tendrá un tiempo para responderse de 8 segundos. En caso que los usuarios respondan de manera correcta a una pregunta sumará 10 puntos. Además se le sumará como puntos la cantidad de segundos que le faltan para alcanzar los 8 segundos que requiere la pregunta.

Se tomará como tiempo de respuesta cuando el usuario presione el botón “enviar”. (enviando la respuesta seleccionada). Por ejemplo: Si la pregunta 1 la respondo

bien y de los 8 segundos que tengo en esa pregunta me sobraron 3 segundos, tendría 13 puntos (10 de la respuesta correcta + 3 de los segundos sobrantes). Si la respuesta es incorrecta no suma puntos. Finalizadas las 5 preguntas, se debe calcular un podio entre los usuarios. Se contempla primer, segundo y tercer lugar. Siendo el primer lugar para el usuario con mayor puntos acumulados.”

2.4 Prompt técnico

A continuación, se muestra el prompt técnico definido según los requisitos mencionados:

LENGUAJE DE PROGRAMACIÓN:

- Python

PLATAFORMA:

- Windows

FUNCIONALIDAD A IMPLEMENTAR:

- Desarrollar una app que permita participar de preguntas a usuarios comunicados por socket de red, calculando los puntos obtenidos y establecer un podio de ganadores. Los jugadores deben ser manipulados por el procesador a través de hilos.

REQUISITOS ESPECÍFICOS:

- Debe tener una pantalla de ingreso de nombre y botón ingresar.
- Debe tener una pantalla de sala de espera de ingreso de los usuarios.
- Debe tener una pantalla para cada pregunta.
- Debe tener una pantalla de finalización donde muestre el podio con los 3 mejores puntajes.
- La pantalla que corresponde a las preguntas debe tener:
 - Un contador decreciente, que es el tiempo para responder la pregunta.
 - Tiene 4 botones que corresponden a las posibles opciones.
 - Los puntos acumulados del participante.
- El cálculo de puntos por pregunta es 10 si la respondió bien más los segundos sobrantes del tiempo en la pregunta, si responde mal no suma puntos.
- Luego de terminar todas las preguntas se muestra el nombre de los 3 participantes con más puntos y los puntos totales de esos 3 participantes

RESTRICCIONES:

- La cantidad máxima de participantes es 10 .
- La cantidad de preguntas son 5.
- La cantidad de opciones en cada pregunta son 4 .
- La cantidad de segundos para cada pregunta es 8.

Fig. 1 Prompt técnico utilizado

2.5 Resultado inicial

Al insertar el prompt técnico en esta herramienta, en la primera iteración se obtuvo un programa funcional. El mismo respetó gran parte de los requisitos expuestos en el prompt, sin embargo, se detectó que uno de ellos no fue interpretado correctamente. El requisito en cuestión es el siguiente:

“El cálculo de puntos por pregunta es 10 si la respondió bien más los segundos sobrantes del tiempo en la pregunta, si responde mal no suma puntos.”

La herramienta interpretó que los segundos sobrantes, en caso de respuesta correcta, debían sumarse únicamente al usuario que contestó más rápido, sin contemplar el tiempo restante de los demás participantes y sumarlo a sus respectivos puntajes, en caso de corresponder. Esto difiere de lo esperado, ya que el objetivo era que cada usuario que respondiera correctamente obtuviera sus puntos más los segundos sobrantes correspondientes.

2.6 Iteraciones

Se buscó resolver el requisito que inicialmente no había sido implementado correctamente por la herramienta, correspondiente al cálculo de puntos de los usuarios. Asimismo, se incorporaron algunas mejoras orientadas a la eficiencia e interfaz de la aplicación. Se detallan a continuación:

- Para los usuarios que respondan correctamente, se deberá sumar, además de los 10 puntos de la pregunta correcta, el tiempo restante para completar la pregunta.
- Agrandar un poco más las ventanas de interfaz de cliente.
- No confiar en el cliente y que sea el servidor el que calcule el tiempo de respuesta.

Fig. 2 Correcciones en iteraciones

Finalmente, luego de cuatro iteraciones se obtuvo un programa funcional que respeta todos los requisitos expuestos en el prompt técnico junto con las mejoras propuestas.

2.7 Resultados finales

Una desventaja del proceso fue la presentación del código. La IA generó un único bloque de texto, usando comentarios para delimitar los archivos `servidor.py` y `cliente.py`. Esto requirió una reestructuración manual para crear un proyecto funcional con archivos separados, un paso necesario para el desarrollo y mantenimiento de software en el mundo real.

Respecto a la interfaz de interacción con el cliente, se observa que es demasiado pequeña no dando posibilidad de manera cómoda su manipulación y visualización de los ítems.

3. Análisis y métricas

El presente desarrollo toma como caso de estudio una aplicación de juego de preguntas y respuestas generada a partir de un prompt técnico detallado procesado por un modelo de lenguaje avanzado (ChatGPT-5). El código

resultante, compuesto por un servidor (`servidor.py`) y un cliente (`cliente.py`), sirve como ejemplo práctico para analizar la implementación de conceptos de programación concurrente y los desafíos vinculados a su diseño.

Con el fin de evaluar la calidad técnica del código obtenido, se aplicaron diversas métricas de ingeniería de software orientadas a caracterizar la estructura, mantenibilidad y desempeño del código. Entre ellas se incluyen las métricas de complejidad ciclomática, acoplamiento, mantenibilidad, escalabilidad y eficiencia de sincronización.

Importancia de las métricas clásicas de software

- Complejidad ciclomática:** mide el número de caminos independientes que existen en el flujo de control de un programa. Su análisis permite estimar el esfuerzo de prueba necesario y la propensión del código a errores lógicos. Cuanto mayor sea la complejidad, menor es la legibilidad y mayor la dificultad de mantenimiento.
- Acoplamiento:** evalúa el grado de dependencia entre los módulos del sistema. Un bajo acoplamiento favorece la modularidad, la reutilización del código y la facilidad de realizar cambios sin afectar otras partes del sistema.
- Mantenibilidad:** determina la facilidad con la que el código puede modificarse, corregirse o extenderse. Es fundamental para proyectos que evolucionan en el tiempo o requieren adaptaciones frecuentes, especialmente cuando el código fue generado con asistencia de herramientas automatizadas.
- Escalabilidad:** analiza la capacidad del sistema para adaptarse a un aumento en la carga de trabajo o en la cantidad de usuarios, sin comprometer su rendimiento ni estabilidad. En aplicaciones concurrentes, esta métrica resulta esencial para evaluar el diseño de los mecanismos de comunicación y sincronización.
- Eficiencia de sincronización:** permite valorar cómo se gestionan los recursos compartidos y la coordinación entre hilos o procesos. Un diseño eficiente minimiza los bloqueos y las condiciones de carrera, mejorando el rendimiento global del sistema.

3.1 Mantenibilidad

De todas ellas, en este trabajo de investigación se desarrolla con mayor profundidad la métrica de *mantenibilidad*, dado que resulta clave para comprender la calidad del código generado por una inteligencia artificial y

su viabilidad a largo plazo dentro de un entorno de desarrollo humano.

El análisis de *mantenibilidad* incluirá la revisión de factores como la claridad del código, modularidad, documentación interna, y capacidad de extensión o refactorización, con el fin de estimar el esfuerzo necesario para realizar modificaciones posteriores sin introducir errores:

- a. En cuanto a claridad del código, el desarrollo presenta una estructura legible y organizada. Esto reduce la carga cognitiva del desarrollador, permitiéndole entender rápidamente el propósito de cada componente sin necesidad de una documentación exhaustiva. Se observa consistencia y claridad en la elección de identificadores, lo que contribuye a la autodescripción del código. Los nombres de funciones y variables reflejan de manera precisa su propósito
- b. Respecto a la modularidad, la lógica está segmentada en funciones con responsabilidades bien definidas. Por ejemplo, la función `send_json` se encarga exclusivamente de serializar y enviar datos, mientras que `broadcast` se ocupa de la difusión de mensajes. Esta separación permite modificar o depurar una parte del sistema sin afectar a las demás.
- c. Más allá de tener una buena legibilidad en el código, en términos de documentación interna, carece de descripciones formales en las funciones, lo que podría afectar su comprensión en proyectos colaborativos. La incorporación de docstrings y comentarios explicativos mejoraría significativamente este aspecto.
- d. Finalmente, en cuanto a la capacidad de extensión o refactorización, tiene varios matices. El diseño modular y la separación de responsabilidades facilitan la incorporación de nuevas características, como más preguntas, cambios en el tiempo de respuesta o ajustes en la interfaz. Sin embargo, el proyecto tiene dos debilidades principales que comprometen su mantenibilidad a largo plazo: Alto acoplamiento y Violación del Principio de Responsabilidad Única. En lo que refiere a Alto acoplamiento, las funciones `client_thread` y `game_thread` están fuertemente acopladas porque ambas operan sobre el mismo conjunto de variables globales, lo que puede atraer problemas de comprensión y razonamiento de la lógica, o condiciones de carrera difíciles de depurar. Por el lado de la violación del Principio de Responsabilidad Única, la función `game_thread` en el servidor es

demasiado grande y hace demasiadas tareas. Es a la vez un gestor de la sala de espera, el director del ciclo de juego, el procesador de respuestas y el calculador del podio. Si se quisiera añadir una nueva característica, como un "descanso" entre partidas o un sistema de revancha, habría que modificar esta función monolítica, aumentando el riesgo de introducir errores en lógicas no relacionadas.

La *mantenibilidad* del sistema es **moderada**. Para su tamaño y alcance actual, el código es comprensible y relativamente fácil de gestionar. Sin embargo, su arquitectura no es robusta frente al crecimiento.

3.2 Métricas complementarias

Dado que el presente trabajo de investigación se enmarca dentro de la técnica de *Vibecoding*, es relevante considerar métricas complementarias que permitan evaluar no sólo la calidad técnica del código, sino también la efectividad del proceso colaborativo entre el programador y la inteligencia artificial generativa. En este contexto, inspirados en los enfoques de evaluación de generación automática de código [7], definimos cuatro métricas específicas:

- a. **Corrección del código IA:** mide cuántas veces el código generado por la IA requirió ajustes antes de compilar o pasar las pruebas. Refleja la precisión inicial del modelo y su capacidad para interpretar correctamente las especificaciones del desarrollador.
- b. **Número de iteraciones:** cuantifica las modificaciones introducidas tras la generación inicial, permitiendo identificar patrones de mejora o fallas recurrentes en la interpretación de instrucciones.
- c. **Reutilización IA efectiva:** evalúa qué porcentaje del código generado por la IA se mantuvo sin cambios significativos en la versión final del sistema. Esta métrica mide el grado de aprovechamiento real del aporte de la IA y la calidad de su integración dentro del flujo de trabajo del programador.
- d. **Velocidad de entrega:** estima el tiempo transcurrido entre la generación del prompt y la obtención de un código funcional. Representa un indicador del impacto de la asistencia IA en la reducción de tiempos de desarrollo y la optimización de iteraciones.

Dentro de este conjunto, la métrica seleccionada para su desarrollo en este trabajo es la *Reutilización IA efectiva*,

por considerarse un indicador clave de la eficiencia del proceso de co-creación.

Analizarla permitirá identificar en qué medida el código sugerido por el modelo se mantuvo estable a lo largo de las iteraciones y cuánto del producto final puede atribuirse directamente al aporte de la inteligencia artificial. Para su cálculo, se realizará una comparación entre la versión inicial del código generada por la herramienta y la versión final obtenida tras las modificaciones. De este modo, se determinará la proporción de líneas o bloques de código que se conservaron sin modificaciones sustanciales respecto a la versión original propuesta por el modelo.

La IA generó una sólida estructura base para el juego cliente-servidor, incluyendo la lógica de sockets, hilos y el flujo general de la aplicación. Esta base fue altamente reutilizable y se conservó en gran medida durante todo el proyecto.

A lo largo de las iteraciones, el modelo demostró una gran capacidad de adaptación progresiva. Integró con éxito mejoras clave —como el cálculo de puntos por tiempo y la validación de respuestas en el servidor— sin romper el código preexistente.

El proceso mantuvo una coherencia funcional, ya que la IA fue capaz de reutilizar las secciones estables del código sin reescribir la lógica principal, optimizando así el esfuerzo de desarrollo.

Finalmente, la efectividad global de la reutilización se demostró en la reducción del tiempo de desarrollo y la estabilidad del producto final. El modelo combinó de manera inteligente la base inicial con las correcciones posteriores, evidenciando un proceso de aprendizaje y reutilización eficiente.

Esto indica que la IA no solo entregó código, sino un sistema coherente y moderadamente extensible. El rol del programador no fue el de escribir desde cero, sino el de auditar, refinar y guiar la base generada, validando el enfoque de *Vibecoding* como un proceso de colaboración aceptablemente eficiente.

4. Conclusiones

Este trabajo de investigación ha demostrado la viabilidad de utilizar un modelo de *Inteligencia Artificial Generativa* como herramienta de co-creación para desarrollar una aplicación de software concurrente funcional. A través de un caso de estudio práctico —un juego de preguntas y respuestas cliente-servidor—, se ha validado un flujo de trabajo donde el rol del programador evoluciona desde la escritura de código hacia la auditoría, el refinamiento y la guía estratégica.

El análisis técnico del producto final reveló una implementación exitosa de patrones de concurrencia, como

el modelo "un hilo por cliente" y el "productor-consumidor". La solución demostró una alta eficiencia de sincronización mediante el uso correcto de Locks y Queues, aunque su arquitectura presenta una escalabilidad limitada inherente al diseño, siendo adecuada para un número acotado de usuarios. La mantenibilidad del código fue evaluada como moderada, destacando la necesidad de refactorizar el estado global para proyectos de mayor envergadura.

La aplicación de la métrica de Reutilización IA efectiva permitieron cuantificar el valor del aporte de la IA. Se concluye que la IA no solo aceleró el desarrollo inicial proveyendo una buena base de código funcional, sino que también demostró ser una herramienta colaborativa capaz de integrar refinamientos y correcciones de manera coherente.

En resumen, la experiencia valida el proceso de co-creación con IA como un paradigma eficaz para el desarrollo de software. El éxito no radicó en la generación de un código perfecto en la primera iteración, sino en la sinergia entre la capacidad generativa del modelo y la supervisión experta del desarrollador, resultando en un producto final que cumple con los requisitos funcionales y técnicos de manera eficiente.

5. Referencias

- [1] Python Software Foundation. (n.d.). threading — Thread-based parallelism. En The Python Standard Library. Consultado el 8 de octubre de 2025, de <https://docs.python.org/3/library/threading.html>
- [2] Python Software Foundation. (n.d.). queue — A synchronized queue class. En The Python Standard Library. Consultado el 8 de octubre de 2025, de <https://docs.python.org/3/library/queue.html>
- [3] Python Software Foundation. (n.d.). socket — Low-level networking interface. En The Python Standard Library. Consultado el 8 de octubre de 2025, de <https://docs.python.org/3/library/socket.html>
- [4] Tanenbaum, A. S., & Bos, H. (2023). Modern operating systems (5th ed.). Pearson. <https://os.ecci.ucr.ac.cr/slides/Andrew-S.-Tanenbaum-Modern-Operating-Systems.pdf>
- [5] Yetiştiren, G., Özsoy, A., Ayerdem, H., & Tüzün, E. (2023). Evaluating the code quality of AI-assisted code generation tools: An empirical study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. arXiv preprint arXiv:2304.10778. <https://arxiv.org/abs/2304.10778>
- [6] Esteban Carnuccio, Mariano Volker, Carlos Tomalino, Matías Adagio, Dario Hirschfeldt, Andrea Vera (2025). Análisis preliminar de herramientas para Vibecoding aplicadas al desarrollo de software. CoNaIISI 2025.
- [7] Ghosh Paul, D., Zhu, H., & Bayley, I. (2024). Benchmarks and metrics for evaluations of code generation: A critical review. arXiv preprint arXiv:2406.12655. <https://arxiv.org/abs/2406.12655>